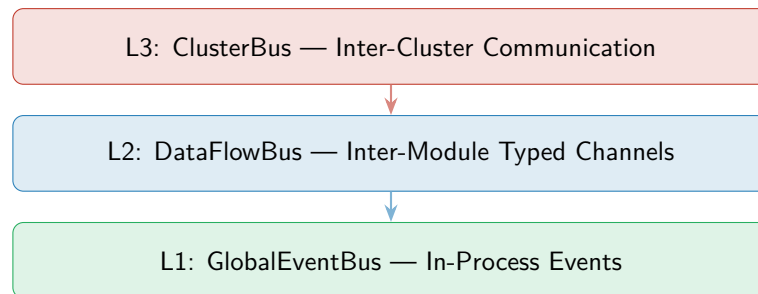


KemdiCode MCP Server

Technical Whitepaper

Version 1.25.0



Dawid Irzyk

Kemdi Sp. z o.o. — dawid@kemdi.pl

<https://kemdi.pl>

February 2, 2026

Licensed under the GNU General Public License v3.0

Contents

1	Abstract	3
2	Introduction	3
2.1	Motivation	3
2.2	Design Principles	3
3	System Architecture	3
3.1	Transport Layer	3
3.2	Tool Registry	5
4	Tool Categories	5
5	3-Layer Bus Architecture	6
5.1	L3: ClusterBus	6
5.2	L2: DataFlowBus	8
5.3	L1: GlobalEventBus	8
5.4	Anti-Amplification Guards	8
6	LLM Magistrale	8
6.1	Aggregation Strategies	8
6.2	PassController	8
7	Cognition System	10
7.1	Store Descriptions	10
7.2	Reactive Event Handlers	11
8	Multi-Agent Coordination	11
9	LLM Provider Integration	11
9.1	Supported Providers	11
9.2	Model Specification Syntax	12
9.3	Structured Output	12
10	Security	12
10.1	Multi-Party Computation	12
10.2	Input Validation	12
11	Knowledge Graph	12
12	Recursive Tool Invocation	13
13	Session Management	13
14	Infrastructure	13
14.1	Redis Schema	13
14.2	Runtime Compatibility	14
14.3	Code Analysis Engine	14
15	Performance Considerations	14

16 Client Compatibility	14
17 Conclusion	14

1 Abstract

KemdiCode MCP Server is an open-source implementation of the Model Context Protocol (MCP) that provides 142 specialized tools across 23 categories for AI-assisted software engineering. The server integrates with multiple IDE clients (Claude Code, Cursor, KiroCode, RooCode) via a stateful HTTP transport layer with Server-Sent Events (SSE). Key innovations include a 3-layer bus architecture for distributed multi-cluster communication, a persistent cognition system enabling AI self-awareness across sessions, multi-agent coordination through Redis-backed shared state, and support for 8 LLM providers with intelligent routing and structured output. This paper describes the architecture, design decisions, and implementation details of version 1.25.0.

2 Introduction

2.1 Motivation

Modern AI-assisted development tools require deep integration between Large Language Models and the developer's working environment. The Model Context Protocol (MCP), introduced by Anthropic, standardizes the interface between AI clients and tool servers. However, reference implementations typically provide a narrow set of tools with limited state management.

KemdiCode MCP Server extends the MCP paradigm with:

- **Comprehensive tooling** — 142 tools covering the full software engineering lifecycle.
- **Persistent cognition** — AI agents retain decision history, confidence calibration, error patterns, and mental models across sessions.
- **Distributed execution** — A 3-layer bus enables multi-cluster LLM orchestration with quality-driven aggregation.
- **Multi-agent coordination** — Redis-backed kanban boards, workspaces, agent registration, and message passing.
- **Provider-agnostic LLM routing** — 8 providers with fallback chains, cost optimization, and structured output via Zod schemas.

2.2 Design Principles

1. **Zero-friction integration.** Every MCP-compatible client works without modification.
2. **Lazy initialization.** Tool schemas and provider connections load on demand.
3. **Fail-safe defaults.** Subsystems degrade gracefully when Redis or LLM providers are unavailable.
4. **Anti-amplification.** Bus bridges enforce hop limits, deduplication, and source guards to prevent feedback loops.
5. **Runtime reconfiguration.** Providers, models, and tool availability can change without server restart.

3 System Architecture

Figure 1 shows the high-level architecture from client to infrastructure.

3.1 Transport Layer

The server exposes an HTTP interface with the following endpoints:

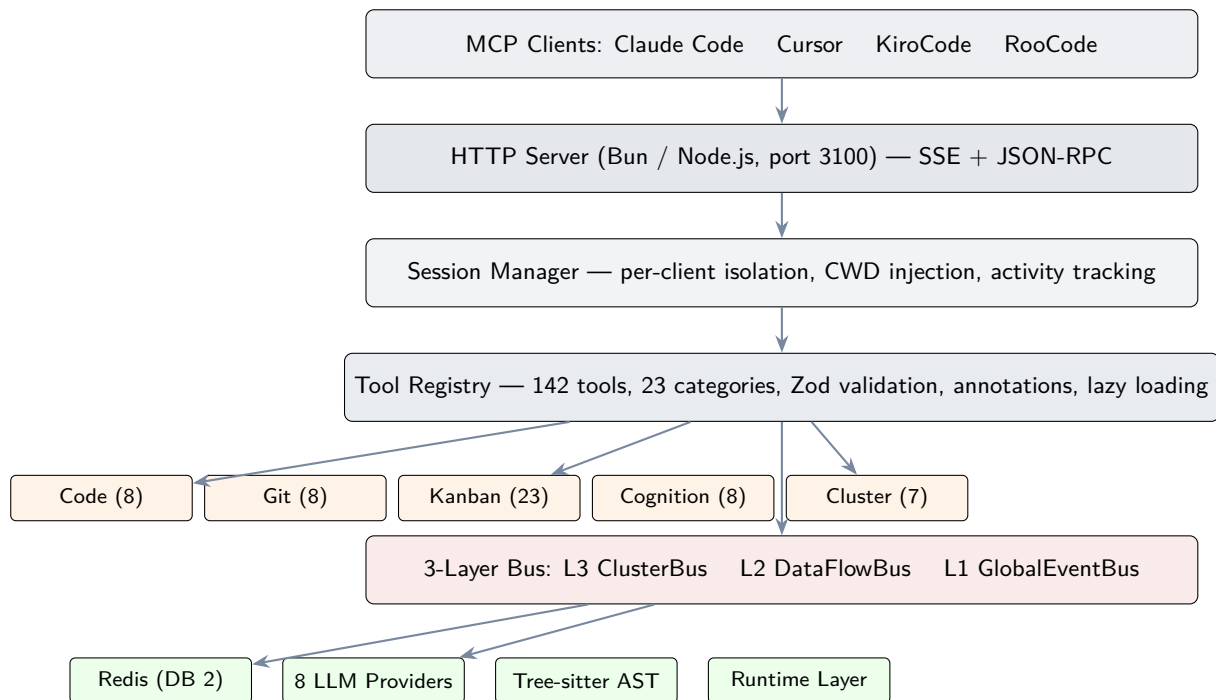


Figure 1: High-level system architecture.

Method	Path	Purpose
GET	/sse	SSE stream for server notifications
POST	/message	JSON-RPC tool invocations
GET	/stream/:id	Per-request execution stream
GET	/health	Server health and uptime
GET	/tools	Tool listing with schemas
GET	/mcp	Streamable HTTP MCP endpoint

Table 1: HTTP endpoints.

Each connected client receives its own session with isolated working directory, SSE keep-alive, and automatic session ID generation. Session state persists in Redis, enabling recovery after disconnection.

3.2 Tool Registry

The tool registry manages 142 tools through a unified `UnifiedTool` interface:

- **Zod schemas** — Every tool defines its input schema using Zod. Schemas are converted to JSON Schema for the MCP protocol, with special handling for `discriminatedUnion` types (flattened into merged object schemas).
- **Annotations** — MCP-level hints (`readOnlyHint`, `destructiveHint`, `openWorldHint`) inform clients about tool behavior.
- **Lazy loading** — Tool implementations load on first invocation. A background warmup phase pre-loads all schemas after server start and emits `tools/list_changed` notifications.
- **Availability checking** — Tools report their health status with `soft/force` modes and suggest fallback alternatives.

4 Tool Categories

Table 2 enumerates all 23 categories with their tool counts and representative tools.

Category	#	Key Tools
Cluster Bus	7	<code>cluster-bus-status</code> , <code>cluster-bus-send</code> , <code>cluster-bus-magistrale</code> , <code>cluster-bus-topology</code>
Cognition	8	<code>decision-journal</code> , <code>confidence-tracker</code> , <code>mental-model</code> , <code>intent-tracker</code> , <code>error-pattern</code> , <code>self-critique</code> , <code>smart-handoff</code> , <code>context-budget</code>
AI Agents	4	<code>plan</code> , <code>build</code> , <code>brainstorm</code> , <code>ask-ai</code>
Multi-LLM	2	<code>multi-prompt</code> , <code>consensus-prompt</code>
Code Analysis	8	<code>code-review</code> , <code>explain-code</code> , <code>find-definition</code> , <code>find-references</code> , <code>semantic-search</code> , <code>code-outline</code>
Line Editing	4	<code>insert-at-line</code> , <code>delete-lines</code> , <code>replace-lines</code> , <code>replace-content</code>
Symbol Editing	3	<code>insert-before-symbol</code> , <code>insert-after-symbol</code> , <code>rename-symbol</code>
Code Modification	5	<code>fix-bug</code> , <code>refactor</code> , <code>auto-fix</code> , <code>auto-fix-agent</code> , <code>write-tests</code>
Project Memory	8	<code>write-memory</code> , <code>read-memory</code> , <code>list-memories</code> , <code>checkpoint-save</code> , <code>checkpoint-restore</code> , <code>checkpoint-diff</code>
Git	8	<code>git-status</code> , <code>git-diff</code> , <code>git-log</code> , <code>git-blame</code> , <code>git-branch</code> , <code>git-add</code> , <code>git-commit</code> , <code>git-stash</code>
File Operations	9	<code>file-read</code> , <code>file-write</code> , <code>file-search</code> , <code>file-tree</code> , <code>file-diff</code> , <code>file-delete</code> , <code>file-move</code> , <code>file-copy</code>

Category	#	Key Tools
Project	5	project-info, run-script, run-tests, run-lint, check-types
Kanban Tasks	12	task-create, task-list, task-update, task-claim, task-assign, task-cluster, task-complexity
Kanban Workspaces	5	workspace-create, workspace-list, workspace-join, workspace-leave, workspace-delete
Kanban Boards	6	board-create, board-list, board-share, board-members, board-invite, board-delete
Recursive	4	invoke-tool, invoke-batch, invocation-log, agent-orchestrate
Multi-Agent	14	agent-list, agent-register, agent-watch, agent-inject, shared-thoughts, get-shared-context
Pipeline	1	pipeline
Session	6	session-list, session-info, session-create, session-recover
MCP Client	3	client-sampling, client-elicited, client-roots
Knowledge Graph	4	graph-query, graph-find-path, loci-recall, sequence-recommend
Thinking	1	thinking-chain
MPC Security	4	mpc-split, mpc-distribute, mpc-reconstruct, mpc-status
RL Learning	2	rl-reward-stats, rl-dopamine-log
System	8	env-info, memory-usage, ai-config, ai-models, tool-health, config, ping, help

Table 2: Tool categories in KemdiCode MCP v1.25.0.

5 3-Layer Bus Architecture

The bus architecture provides communication at three distinct scopes, connected by anti-amplification bridges. Figure 2 illustrates the layered design.

5.1 L3: ClusterBus

The ClusterBus enables inter-cluster communication over Redis Pub/Sub. It supports 12 signal types grouped into four namespaces:

Three send modes are supported: **unicast** (target a specific node), **broadcast** (all nodes), and **routed** (via MetaRouter tag-based rules with AND/OR logic). The **SignalFlowController** provides backpressure, rate limiting, priority filtering, circuit breaking, bloom filter deduplication, and HMAC authentication.

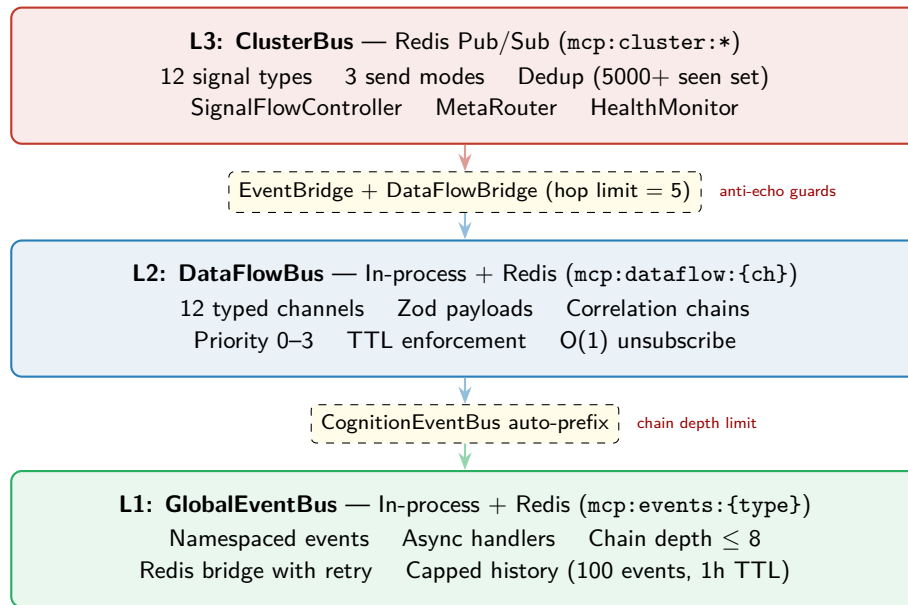


Figure 2: 3-layer bus architecture with anti-amplification bridges.

Namespace	Signal Types
llm:*	request, result, stream-chunk, error
cluster:*	join, leave, heartbeat
data:*	broadcast, unicast
control:*	pause, resume, config

Table 3: ClusterBus signal types.

5.2 L2: DataFlowBus

The DataFlowBus provides 12 typed channels with Zod-validated payloads for inter-module communication:

- **AI channels:** `ai:completion`, `ai:structured`, `ai:research`
- **Kanban channels:** `kanban:task-change`, `kanban:complexity`
- **Cognition channels:** `cognition:decision`, `cognition:intent`, `cognition:error`
- **Agent channels:** `agent:status`, `agent:message`
- **System channels:** `system:health`, `system:config`

Each message is wrapped in a `DataFlowEnvelope` carrying metadata: correlation ID, schema version, priority (0–3), TTL, source/target filters, session ID, and agent ID.

5.3 L1: GlobalEventBus

The GlobalEventBus is an in-process singleton with namespaced events (pattern: `module:category:action`). Handlers execute asynchronously via `queueMicrotask` with configurable priority. A maximum chain depth of 8 prevents cascading event storms. The Redis bridge propagates events across sessions with 2-retry backoff and capped history.

5.4 Anti-Amplification Guards

Bridges between layers employ four mechanisms to prevent feedback loops:

Guard	Location	Mechanism
Hop limit	bridges.ts	<code>MAX_BRIDGE_HOPS = 5</code> ; signals carry <code>_bridgeHops</code> counter
Source prefix	bridges.ts	Skip messages with <code>source.startsWith("cluster:")</code>
Dedup	bus.ts	ClusterBus: seen set (5000+); DataFlowBus: message ID set
Chain depth	global-bus.ts	<code>MAX_EVENT_CHAIN_DEPTH = 8</code> on nested emits

Table 4: Anti-amplification guards.

6 LLM Magistrale

The LLM Magistrale orchestrates distributed prompt execution across multiple clusters. Figure 3 shows the dispatch flow.

6.1 Aggregation Strategies

6.2 PassController

The PassController governs multi-pass LLM execution within a single cluster:

- **min-passes** — the LLM self-assesses task complexity and declares a minimum pass count (capped to `maxPasses` budget), with early-stop on quality.
- **quality-target** — iterates until quality $\geq$ threshold or budget exhaustion.
- **fixed** — skips assessment, executes exactly N passes.

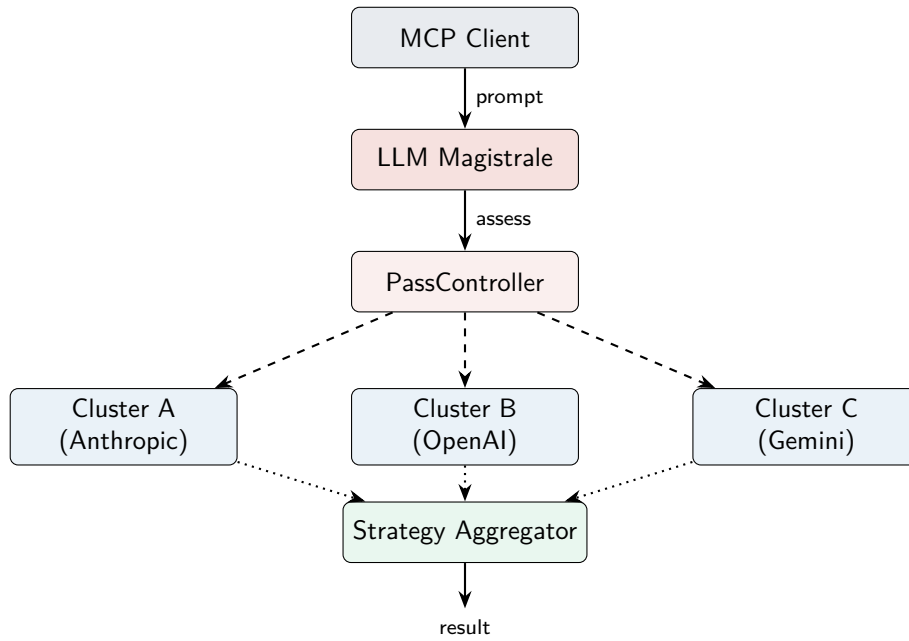


Figure 3: LLM Magistrale dispatch and aggregation.

Strategy	Behavior	Use Case
<code>first-wins</code>	Return first response, cancel remaining	Low-latency queries
<code>best-of-n</code>	Score all responses, pick highest quality	Code generation
<code>consensus</code>	Jaccard similarity scoring, require agreement	Architecture decisions
<code>fallback-chain</code>	Try clusters in order, stop on success	High availability

Table 5: Magistrale aggregation strategies.

7 Cognition System

The cognition layer provides persistent AI self-awareness across sessions via 8 Redis-backed stores, interconnected by a cross-linker. Figure 4 shows the store relationships.

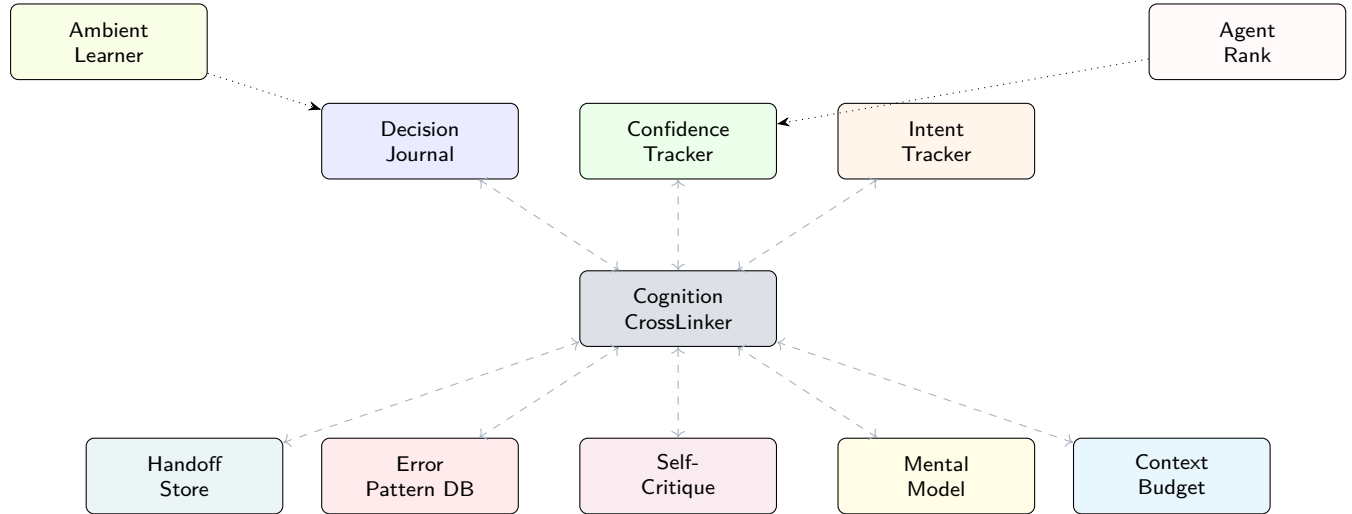


Figure 4: Cognition stores interconnected via CognitionCrossLinker.

7.1 Store Descriptions

DecisionJournal

Records decisions with options, reasoning, and outcome tracking. Supports retrospective evaluation.

ConfidenceTracker

Maintains calibrated confidence scores (0–1) per domain with trend analysis.

IntentTracker

Hierarchical goal tracking with mission, sub-goals, and drift detection.

ErrorPatternDB

Cross-session error database. Matches symptoms to known patterns and suggests fixes.

SelfCritiqueStore

Post-session reflection producing lessons learned. Supports check-application queries.

MentalModelStore

Component-relationship maps for system architecture with dependency chains and invariant checks.

HandoffStore

Auto-enriched session snapshots for structured context transfer between agents.

ContextBudgetManager

Token estimation for context windows with trim suggestions.

AmbientLearner

Silent background knowledge gathering: tool usage sequences, file relationships, time patterns.

AgentRankStore

Composite scoring system with tiers from bronze to diamond, with temporal decay.

7.2 Reactive Event Handlers

The cognition system reacts to events via 9 handlers on the GlobalEventBus:

1. `decision:recorded` → auto-create confidence record + cross-link.
2. `confidence:low` → check intent drift.
3. `error:recorded` → scan recent decisions for causal links.
4. `error:matched` → log known fix suggestion.
5. `critique:lesson-learned` → link to error patterns.
6. `intent:drifted` → trigger self-critique.
7. `handoff:created` → link mental models.
8. `model:stale` → flag affected decisions and intents.
9. `confidence:outcome` → propagate to linked decisions.

8 Multi-Agent Coordination

KemdiCode supports multi-agent workflows where multiple AI sessions collaborate on shared tasks. The coordination infrastructure consists of:

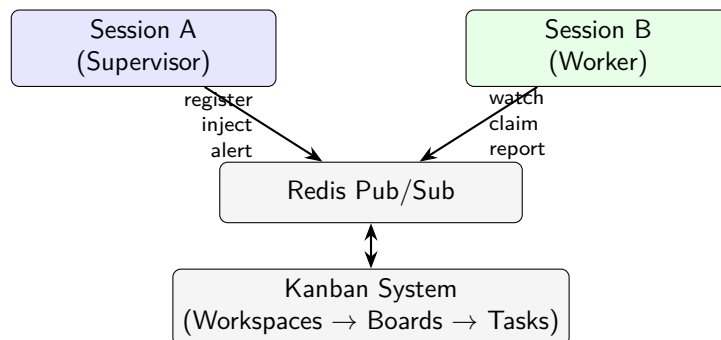


Figure 5: Multi-agent coordination via Redis and kanban.

- **Agent Registry** — agents register with role, capabilities, and status. Discovery via `agent-list`.
- **Message Passing** — `agent-inject` sends context to specific agents; `agent-alert` broadcasts to all.
- **Shared Thoughts** — collective knowledge accessible by all agents via `shared-thoughts`.
- **Kanban Boards** — workspaces with role-based membership (owner/admin/member/viewer), boards, and tasks with claiming, assignment, comments, and LLM-driven clustering.
- **Task Complexity** — AI-powered estimation of task difficulty with priority scoring.

9 LLM Provider Integration

9.1 Supported Providers

KemdiCode integrates 8 LLM providers through a unified routing layer:

Provider	Alias	SDK	Notes
Anthropic	a	Native	Claude models, extended thinking tokens
OpenAI	o	Native	GPT-4, o1, structured output
Gemini	g	Native	Gemini Pro/Flash/Ultra
Groq	q	Compat.	Fast inference, Llama/Mixtral
DeepSeek	d	Compat.	DeepSeek-V3, R1
Ollama	l	Compat.	Local models, zero cost
OpenRouter	r	Compat.	Unified gateway to 100+ models
Perplexity	p	Compat.	Sonar search-augmented models

Table 6: LLM providers. “Compat.” denotes OpenAI-compatible adapters.

9.2 Model Specification Syntax

Models are specified as `provider:model:thinking`, for example:

- `a:claude-sonnet-4-5:4k` — Anthropic Claude Sonnet 4.5 with 4K thinking tokens
- `o:gpt-4o` — OpenAI GPT-4o
- `p:sonar-pro` — Perplexity Sonar Pro
- `l:llama3` — Ollama local Llama 3

9.3 Structured Output

The `generateObject()` function produces typed responses using Zod schemas. It employs `jsonrepair` to handle malformed LLM JSON output, providing reliable structured data extraction across all providers.

10 Security

10.1 Multi-Party Computation

KemdiCode includes a Shamir’s Secret Sharing implementation for distributed secret management:

1. `mpc-split` — Split a secret into n shares with threshold t .
2. `mpc-distribute` — Distribute shares to agents via encrypted Redis channels.
3. `mpc-reconstruct` — Reconstruct the secret from $\geq t$ shares.
4. `mpc-status` — Monitor share distribution and reconstruction state.

10.2 Input Validation

- All tool inputs validated via Zod schemas before execution.
- File paths undergo traversal protection (`validatePathSafe()`).
- Redis JSON parsing uses prototype pollution protection (blocked keys: `__proto__`, `constructor`, `prototype`).
- ClusterBus signals authenticated via HMAC.
- MetaRouter regex patterns capped at 200 characters to prevent ReDoS.

11 Knowledge Graph

The Loci subsystem implements a knowledge graph for error-to-solution pathfinding:

- `graph-query` — traverse nodes and edges in the knowledge graph.

- **graph-find-path** — find shortest path between two concepts (e.g., error symptom to fix).
- **loci-recall** — retrieve ambient learning insights.
- **sequence-recommend** — suggest next tool invocation based on historical usage patterns.

The graph is stored in Redis with bidirectional edges, enabling both forward (symptom → cause → fix) and reverse (fix → where applied) traversals.

12 Recursive Tool Invocation

The recursive invocation system allows tools to call other tools, enabling autonomous agent loops:

- **invoke-tool** — Call any registered tool with arguments, respecting depth limits.
- **invoke-batch** — Execute multiple tool calls in parallel with concurrency control.
- **agent-orchestrate** — Autonomous loop: plan → execute → evaluate → repeat.
- **pipeline** — Define sequential/parallel tool chains with data passing between stages.

Safety constraints include configurable maximum recursion depth, per-invocation timeout, and invocation logging for auditability.

13 Session Management

Sessions provide isolated execution contexts per client connection:

- Automatic session ID generation with Redis persistence.
- Per-session working directory (CWD) injection.
- SSE keep-alive with configurable heartbeat interval.
- **Session recovery** (**session-recover**) — single-call restoration of active-session memory, latest handoff, loci resurrection, tool availability, agent rankings, and ambient learning insights.
- Session switching for multi-context workflows.

14 Infrastructure

14.1 Redis Schema

All persistent state uses Redis DB 2 with the following key patterns:

Key Pattern	Purpose
mcp:context:*	Shared tool outputs for multi-agent context
mcp:agents:*	Agent registry, rankings, and status
mcp:messages:*	Inter-agent message queues
mcp:kanban:*	Tasks, boards, workspaces, clusters
mcp:memory:*	Project memory entries
mcp:cognition:*	Decisions, confidence, models, intents, errors, lessons
mcp:cluster:*	Cluster bus signals and registry
mcp:dataflow:*	Typed message channels
mcp:events:*	Event history and propagation
mcp:channel:*	Pub/Sub channel metadata

Table 7: Redis key schema (DB 2).

14.2 Runtime Compatibility

The server runs on both Bun and Node.js runtimes. A runtime abstraction layer (`src/runtime/`) provides unified interfaces for HTTP, process management, and cryptographic operations, with automatic runtime detection.

14.3 Code Analysis Engine

Tree-sitter WASM parsers provide AST-based code analysis for 19 programming languages, enabling symbol navigation, definition finding, reference searching, and safe rename operations across polyglot codebases.

15 Performance Considerations

- **Lazy schema loading** — Tool schemas load on first invocation; background warmup runs after server start without blocking client connections.
- **O(1) unsubscribe** — Both DataFlowBus and HealthMonitor use Map-based subscription tracking for constant-time cleanup.
- **Circuit breakers** — SignalFlowController implements circuit breaking to prevent cascade failures in the cluster bus.
- **Bloom filter deduplication** — ClusterBus uses a bloom filter for probabilistic dedup, with a secondary seen-set for exactness.
- **Backpressure** — Signal flow control with queue depth monitoring, buffer/drop policies, and priority-based filtering.
- **Idle cleanup** — DataFlowBus subscriptions expire after 60 minutes of inactivity; HealthMonitor prunes stale nodes automatically.

16 Client Compatibility

KemdiCode MCP Server is fully compatible with all major MCP clients:

IDE / Editor	Status
Claude Code	Fully supported (primary)
Cursor	Fully supported
KiroCode	Fully supported
RooCode	Fully supported

Table 8: Client compatibility matrix.

17 Conclusion

KemdiCode MCP Server v1.25.0 demonstrates that the Model Context Protocol can serve as a foundation for comprehensive AI-assisted software engineering. By combining 142 specialized tools, a 3-layer distributed bus architecture, persistent cognition, and multi-provider LLM orchestration, the server transforms MCP from a simple tool-calling interface into a full-featured AI development platform.

The architecture prioritizes safety through anti-amplification guards, input validation, HMAC authentication, and prototype pollution protection. Lazy initialization and runtime reconfiguration ensure the server remains responsive under varying workloads and configurations.

Future work includes expanding the knowledge graph with cross-project learning, adding WebSocket transport support, and deepening the reinforcement learning subsystem for autonomous tool selection optimization.

KemdiCode MCP Server is open-source software licensed under the GNU General Public License v3.0.

Source code: git.kemdi.pl

Author: Dawid Irzyk — dawid@kemdi.pl — Kemdi Sp. z o.o.